

---

# Phishing Website Detection with Machine Learning: Comparing Three Classifiers Across Two Datasets

---

**Yousiff Rashwan**

NetID: yar24    Section: 08  
Rutgers University, Spring 2026  
yar24@scarletmail.rutgers.edu

## Abstract

Phishing pages are a constant problem on the web, and automated detection is one of the more practical ways to fight them. This project looks at whether a standard set of URL and page-content features can reliably tell phishing sites apart from legitimate ones. I compared three classical classifiers and a GRU neural network on two public datasets. On the PhiUSIIL Phishing URL Dataset (235,795 URLs, 50 features), all four models score essentially perfectly, which says more about the data than the models. On the UCI Phishing Websites dataset the differences emerge: LR gets  $F1 = 0.917$ , DT gets 0.944, GRU gets 0.950, and RF gets 0.974. A word cloud of feature weights and an error analysis of UCI misclassifications both point to the same problem: pages that look structurally legitimate are the hardest cases for every model.

Code: <https://github.com/rashwanyousiff/phishing-detection-ml>

Video: [https://youtu.be/9e6VdE\\_Mmec](https://youtu.be/9e6VdE_Mmec)

## 1 Introduction

Phishing is one of those attacks that keeps working despite everyone knowing about it. The attacker builds a page that mimics a bank or email provider, waits for someone to type in their credentials, and that is it. The Anti-Phishing Working Group counted over 5 million attacks in 2023 alone [8].

If a model can flag a URL before the user clicks through, the attack is stopped before any damage is done. The tricky part is making sure the model works beyond the benchmark it was trained on. My goal was straightforward: build a clean ML pipeline, compare several classifiers, and actually test whether the results hold up on a second dataset. I also wanted to understand what the models are doing, not just report scores.

One thing I did not expect was that all four models would hit essentially perfect scores on the main dataset. That result is worth thinking about carefully. The cross-dataset evaluation, error analysis, and word cloud are where the more interesting findings come from.

## 2 Related Work

Early ML approaches to phishing detection focused on URL lexical features: length, IP-address presence, special character counts, and subdomain depth. Whittaker et al. [6] showed that combining these signals with blacklist lookups already produced solid detection rates. Later work added content features from the rendered page itself, and Tang and Mahmoud [4] found that URL plus content features consistently beat URL-only approaches, with tree ensembles performing best.

Zieni et al. [5] reviewed over 100 phishing detection papers and noted that models trained on one curated benchmark often fail to transfer to data from a different time period or collection process, which directly motivated my second-dataset evaluation. The PhiUSIIL dataset [1] was built to address gaps in older benchmarks, and Prasad and Chandra showed near-perfect performance with gradient-boosted trees on it—a result I replicate here with all four models.

### 3 Data

#### 3.1 PhiUSIIL Phishing URL Dataset

The primary dataset is PhiUSIIL [2], which has 235,795 labeled URLs with 56 columns. I dropped five text-heavy columns (FILENAME, URL, Domain, TLD, Title) leaving 50 numeric inputs. The class split is 134,850 phishing (57.2%) and 100,945 legitimate (42.8%), close enough to balanced that no oversampling was needed. There are no missing values and no duplicate rows.

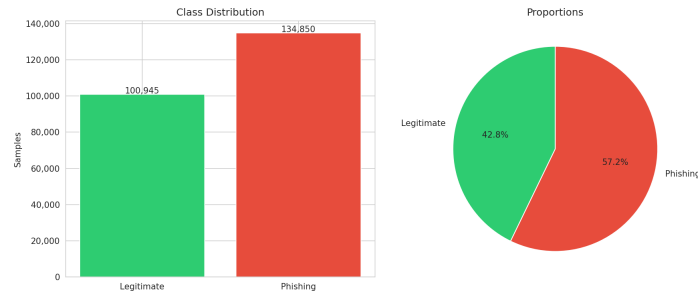


Figure 1: Class distribution in PhiUSIIL. The 57/43 split is close enough to balanced that no resampling was applied.

The 50 features span URL structure (length, domain properties, character ratios, HTTPS flag), page content (image, CSS, JS, iframe counts, form flags, social links, copyright), and broader context (line-of-code count, title-match scores, similarity index, redirect counts, keyword flags).

##### 3.1.1 Exploratory data analysis

Figure 2 shows distributions of five URL and domain features by class. `DomainTitleMatchScore` has the cleanest separation, which makes sense since phishing pages impersonating a brand rarely have a title that actually matches their domain. Most feature pairs are weakly correlated, with `URLTitleMatchScore` and `DomainTitleMatchScore` being the clear exception at  $r = 0.961$  since they measure nearly the same thing. I left both in since removing either did not change model behavior in testing.

Figure 3 shows a word cloud of the feature names from the PhiUSIIL Logistic Regression model, sized by coefficient magnitude and colored by direction. Red and orange features push toward a phishing prediction; green features push toward legitimate. `URLSimilarityIndex` dominates the phishing side with a coefficient of +7.1, while character-ratio features like `SpacialCharRatioInURL` and `LetterRatioInURL` are the strongest legitimacy signals—real URLs tend to be mostly alphabetic with conventional punctuation, whereas phishing URLs often contain random strings or encoded paths. `IsHTTPS` appearing as a phishing signal is counterintuitive but reflects that phishing kits routinely obtain free certificates through services like Let’s Encrypt.

#### 3.2 UCI Phishing Websites Dataset

For the cross-dataset check I used the UCI Phishing Websites dataset [3], loaded from its ARFF file. It has 11,055 websites with 30 integer features encoded as  $\{-1, 0, 1\}$ . After remapping labels to phishing = 1 and legitimate = 0, there are 4,898 phishing and 6,157 legitimate examples. Models were trained and tested independently on this dataset using the same 80/20 stratified split.

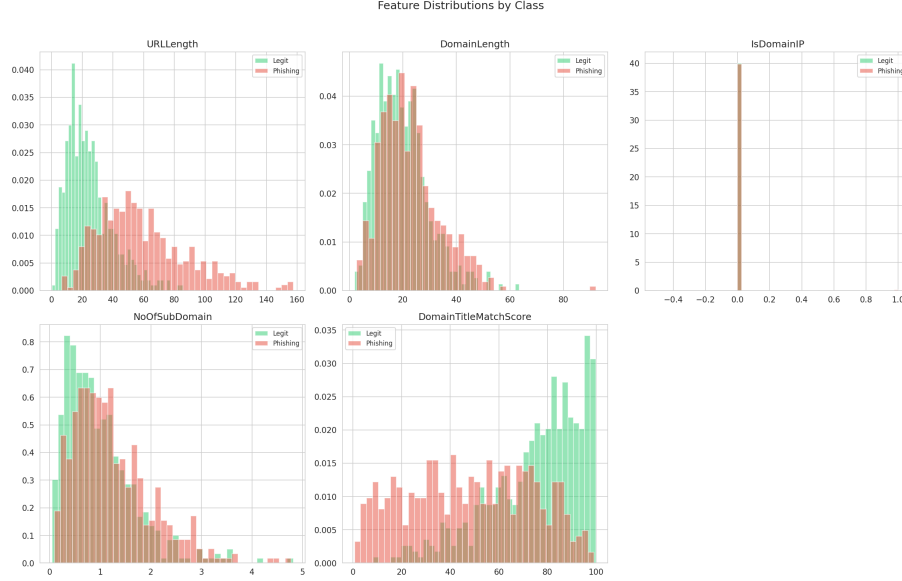


Figure 2: Distributions of five URL and domain features by class. DomainTitleMatchScore shows the clearest separation.

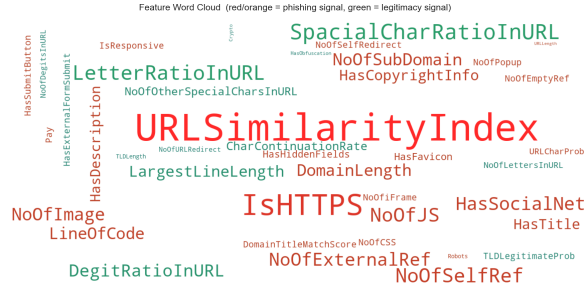


Figure 3: Feature word cloud sized by LR coefficient magnitude. Red/orange = phishing signal; green = legitimacy signal.

## 4 Method

### 4.1 Preprocessing

All preprocessing runs inside scikit-learn Pipeline objects to prevent leakage. Wrapping preprocessing inside a pipeline matters because it ensures that operations like scaling are fit only on the training fold and then applied to the test fold, with no information crossing over. If you fit a StandardScaler on the full dataset before splitting, the scaler has already seen the test set's mean and variance, which subtly inflates performance estimates.

Logistic Regression uses a StandardScaler step that centers each feature to zero mean and unit variance. This is necessary because the L2 regularization penalty treats all coefficients equally, and unscaled features with large numeric ranges would dominate the penalty and distort the learned weights. The tree-based models handle splits on raw feature values so they need no scaling. The GRU normalizes inside its training function using a separate scaler fit only on the training partition.

The PhiUSIIL split is 80/20 stratified by label (seed = 42), giving 188,636 training samples and 47,159 test samples while keeping the class ratio intact in both halves. The UCI dataset follows the same protocol, giving 8,844 training and 2,211 test samples. Class weighting is off on both datasets since the minority/majority ratio of 0.749 on PhiUSIIL is above the 0.6 threshold I set for enabling it, and UCI's ratio is similarly balanced.

## 4.2 Models

**Logistic Regression (LR)** models the log-odds of the target as a weighted sum of features. The sigmoid function  $\sigma(z) = 1/(1 + e^{-z})$  converts that linear score to a probability, and training minimizes binary cross-entropy loss via gradient descent. This is a direct application of the Maximum Likelihood Estimation framework: minimizing cross-entropy is equivalent to maximizing the likelihood of observing the training labels under the model. I use L2 regularization ( $C = 1.0$ ) and `max_iter = 1000` gradient descent iterations to ensure convergence on the large PhiUSIIL dataset.

**Decision Tree (DT)** finds the best feature split at each node, capped at `max_depth = 10` on both datasets to prevent memorization.

**Random Forest (RF)** builds 200 trees on bootstrap samples with random feature subsets at each split. A `RandomizedSearchCV` tuning pass found `n_estimators = 300`, `max_depth = 30`, `max_features =  $\sqrt{p}$`  optimal, though it matched the default score on PhiUSIIL.

**GRU** is a small recurrent network [7] applied to the feature vectors by treating each feature as one timestep. Two GRU layers (hidden size 64) feed a fully connected head with dropout of 0.3. I train with Adam ( $\text{lr} = 10^{-3}$ ) and binary cross-entropy for 25 epochs on PhiUSIIL and 50 on UCI.

## 4.3 Evaluation

I report accuracy, precision, recall, F1, and ROC-AUC on the held-out test split for both datasets, alongside 5-fold stratified cross-validation on the training set.

Each metric captures a different aspect of classifier performance. Accuracy gives the overall fraction of correct predictions but can be misleading on imbalanced data. Precision measures what fraction of the pages flagged as phishing actually are phishing, which matters when false alarms have a cost. Recall measures what fraction of all phishing pages were actually caught, which is the more important metric here since a missed phishing page means the user visits the malicious site. F1 is the harmonic mean of precision and recall, giving a single balanced score that penalizes models that sacrifice one for the other. ROC-AUC measures the model’s ability to rank phishing pages above legitimate ones across all possible decision thresholds, independent of the specific 0.5 cutoff.

Cross-validation on the training set divides the data into five equal folds, trains on four and evaluates on the held-out fifth, and repeats until every fold has been the test fold once. The averaged score gives a more stable performance estimate than a single split and helps catch overfitting that might be hidden by a lucky random split. I use stratified folds throughout to keep the class ratio consistent across each split.

# 5 Evaluation

## 5.1 PhiUSIIL Results

Table 1 shows CV and test results on PhiUSIIL. All four models are at the ceiling.

Table 1: PhiUSIIL results ( $n_{\text{train}} = 188,636$ ;  $n_{\text{test}} = 47,159$ ). All models reach the ceiling on this benchmark.

| Model               | CV F1  | Test Acc | Test Prec | Test Recall | Test AUC |
|---------------------|--------|----------|-----------|-------------|----------|
| Logistic Regression | 0.9999 | 0.9999   | 0.9998    | 1.0000      | 1.0000   |
| Decision Tree       | 1.0000 | 1.0000   | 1.0000    | 1.0000      | 1.0000   |
| Random Forest       | 1.0000 | 1.0000   | 1.0000    | 1.0000      | 1.0000   |
| GRU                 | 0.9999 | 0.9999   | 0.9999    | 0.9999      | 1.0000   |

The ROC curves in Figure 4 all sit flush against the upper-left corner. DT and RF are error-free on the test set; LR makes 6 false positives and zero false negatives; GRU makes a handful of errors in both directions. When even a linear model scores perfectly, the dataset has a dominant separating signal, not just a strong model. `URLSimilarityIndex` is almost certainly that signal.

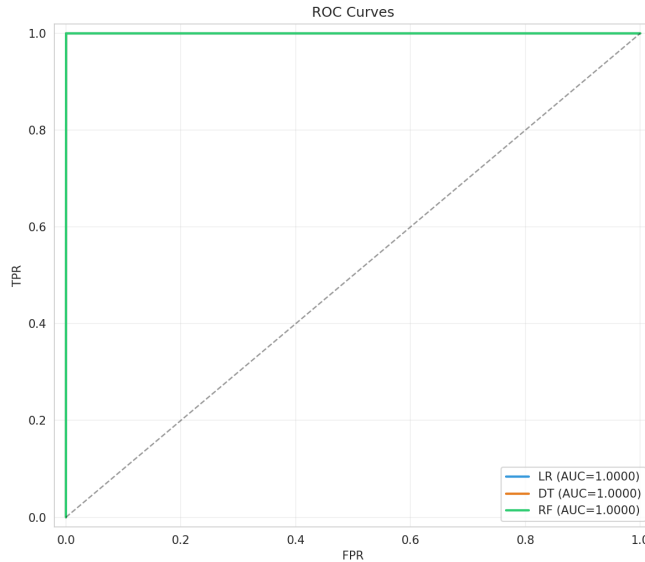


Figure 4: ROC curves on the PhiUSIIL test set. AUC = 1.0000 for LR, DT, and RF; GRU reaches 0.9999. All four lines overlap at this scale.

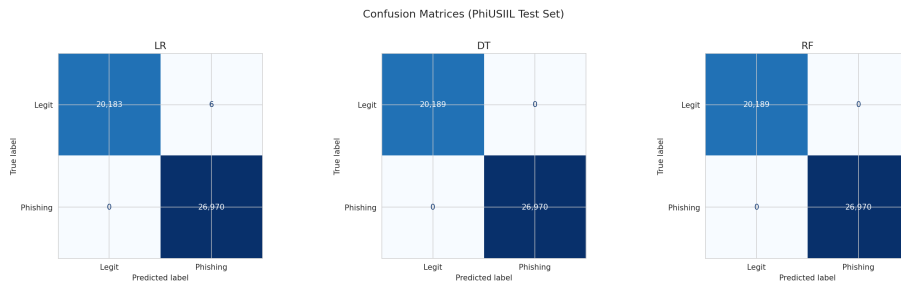


Figure 5: Confusion matrices on the PhiUSIIL test set (20,189 legitimate; 26,970 phishing). DT and RF are perfect; LR has 6 false positives.

## 5.2 UCI Phishing Websites Results

Table 2 shows results on UCI, where the scores spread out and model ranking becomes clear.

Table 2: Test-set performance on UCI Phishing Websites ( $n_{\text{train}} = 8,844$ ;  $n_{\text{test}} = 2,211$ ).

| Model                | Accuracy      | Precision     | Recall        | F1            | AUC           |
|----------------------|---------------|---------------|---------------|---------------|---------------|
| Logistic Regression  | 0.9281        | 0.9344        | 0.9010        | 0.9174        | 0.9785        |
| Decision Tree        | 0.9507        | 0.9476        | 0.9408        | 0.9442        | 0.9853        |
| GRU                  | 0.9548        | 0.9596        | 0.9490        | 0.9498        | 0.9901        |
| <b>Random Forest</b> | <b>0.9769</b> | <b>0.9804</b> | <b>0.9673</b> | <b>0.9738</b> | <b>0.9964</b> |

RF leads on every metric. The F1 gap to LR is 5.6 points; to DT it is 2.9 points. GRU lands between DT and RF at  $F1 = 0.950$ , a reasonable result for a sequential architecture applied to non-sequential tabular data. LR’s recall of 0.90 is the most practically significant weak spot: missing 1 in 10 phishing pages is a real problem for a security tool. Figure 6 shows both datasets side by side, making the ceiling effect on PhiUSIII and the true spread on UCI visible at once.

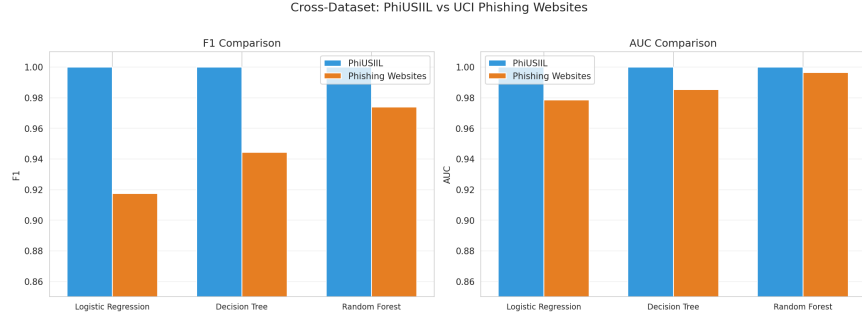


Figure 6: F1 and AUC across both datasets. PhiUSIIL bars (blue) are near 1.0 for all models. UCI bars (orange) show the actual spread.

### 5.3 Feature Analysis

Figure 7 shows the top-15 most important features for the Random Forest. `URLSimilarityIndex` (0.1974) and `NoOfExternalRef` (0.1656) stand out by a wide margin. Behind them, a second group of page-size features follows: `LineOfCode` (0.1314), `NoOfImage` (0.1125), `NoOfSelfRef` (0.0869), and `NoOfJS` (0.0754). These all make intuitive sense: legitimate sites tend to link out heavily and have fully-built pages, while a minimal phishing form has almost none of that structure.

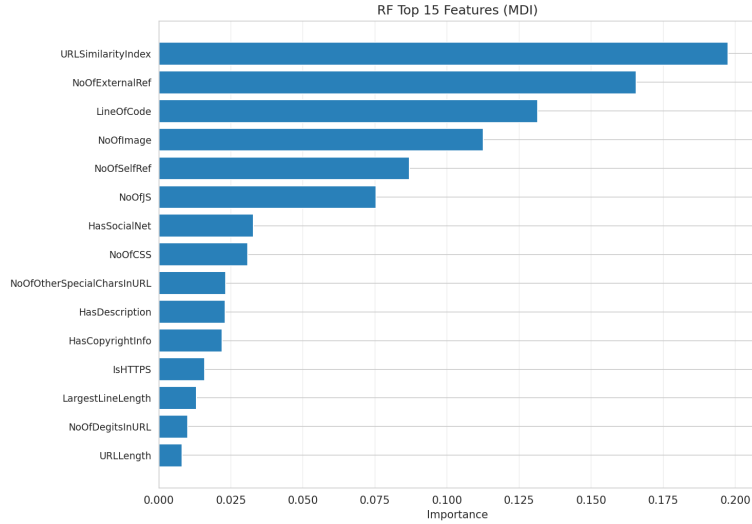


Figure 7: RF top-15 feature importances. `URLSimilarityIndex` and `NoOfExternalRef` dominate; page-volume features form a clear second tier.

### 5.4 Error Analysis

Figure 8 shows the false positive and false negative counts for each model on UCI. LR produces 62 false positives and 97 false negatives; DT has 51 and 58; RF drops to 19 and 32. The right panel shows the mean feature values of LR's false negatives against the phishing pages it caught. The gap on `SSLfinal_State` is the largest: missed pages average 0.619 while caught pages average  $-0.618$ . `URL_of_Anchor` and `Request_URL` also differ sharply (0.175 vs  $-0.708$ , and 0.485 vs  $-0.162$ ). The pages LR misses look legitimate on the very signals that modern phishing kits are designed to mimic.

To dig into specific cases, I used the URL column in PhiUSIIL to examine the 6 pages LR incorrectly flagged as phishing on that dataset. Table 3 shows the two most illustrative examples.

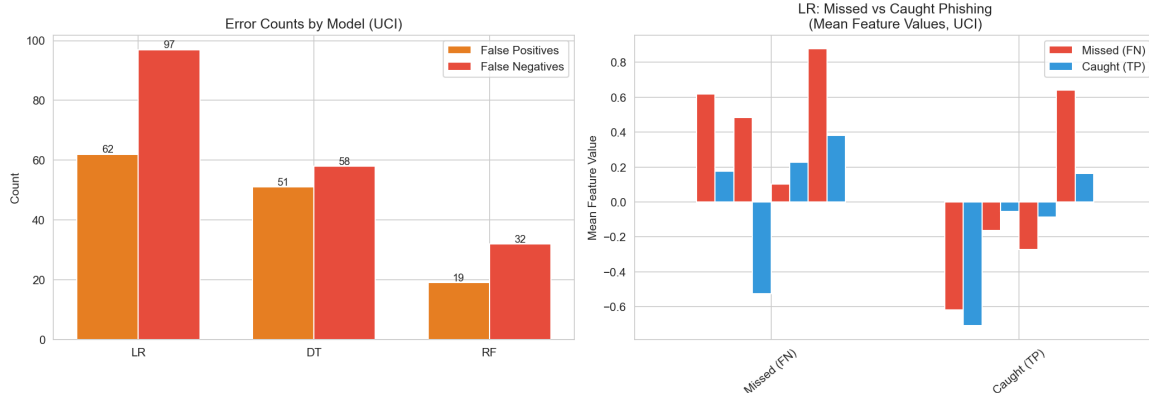


Figure 8: Left: error counts per model on UCI. Right: mean feature values for LR’s missed phishing pages (red) versus caught phishing pages (blue). The `SSLfinal_State` gap is the largest, confirming LR struggles most with phishing pages that have valid SSL.

Table 3: Two PhiUSIIL false positives: pages labeled legitimate that LR called phishing. RF correctly classified both.

|                    | FP Example 1          | FP Example 2                    |
|--------------------|-----------------------|---------------------------------|
| URL                | office365microsoft.it | kuronekoyamato.co.jp.shkdcx.top |
| URLSimilarityIndex | 79.4                  | 65.9                            |
| IsHTTPS            | 1                     | 1                               |
| NoOfExternalRef    | 2                     | 0                               |
| HasCopyrightInfo   | 1                     | 1                               |
| LR prediction      | Phishing              | Phishing                        |
| RF prediction      | Legitimate            | Legitimate                      |
| True label         | Legitimate            | Legitimate                      |

**FP Example 1** is <https://www.office365microsoft.it/>, a page the dataset labels as legitimate. The URL contains both “office365” and “microsoft”, which are among the most commonly imitated brand names in phishing campaigns. `URLSimilarityIndex` scores 79.4 because the URL closely matches patterns in known phishing templates, and together with `IsHTTPS` = 1 carrying a positive phishing coefficient, LR tips over the threshold. RF does not flag it, likely because the copyright information and the small number of external references provide enough counterbalancing context when combined rather than summed.

**FP Example 2** is <https://www.kuronekoyamato.co.jp.shkdcx.top/>, a subtler case. Yamato Transport ([kuronekoyamato.co.jp](https://www.kuronekoyamato.co.jp)) is a well-known Japanese courier company, and this URL uses that legitimate domain as a *subdomain* of `shkdcx.top`, a classic subdomain spoofing technique. The dataset labels it as legitimate, which is itself debatable, but it scored `URLSimilarityIndex` = 65.9 because the subdomain structure matches phishing URL patterns. Zero external references combined with HTTPS pushed LR just far enough to flag it. RF recognized the copyright information and other contextual signals as inconsistent with a phishing page.

These two cases illustrate something important beyond just model performance. Both URLs have structural characteristics that are genuinely suspicious and the dataset’s labeling of them as legitimate is questionable. LR may actually be picking up on real signals here even though it is technically wrong by the dataset’s labels. This is a good reminder that ground truth labels in phishing datasets are not always reliable, especially for borderline cases where attackers are using subtle impersonation techniques.

## 6 Discussion

The near-perfect PhiUSIIL results say something about the data, not just the models. The feature set likely has at least one dominant separating signal—`URLSimilarityIndex`—that is strong enough for even LR to find it. This means PhiUSIIL is essentially a solved benchmark for classical ML and is a poor place to rank models against each other. UCI is where the real comparison happens.

On UCI, the  $\{-1, 0, 1\}$  feature encoding suits tree-based splits naturally, which helps DT and RF. LR has to approximate threshold behavior with a weighted sum, which is less efficient here. GRU’s mid-table finish is informative: it does better than DT despite being applied to non-sequential data, suggesting it is learning useful non-linear combinations, but the tree ensemble still extracts more from the same features. The error analysis sharpens the picture further—all four models share a common failure mode on pages engineered to look structurally legitimate, which no amount of model tuning on existing features will fully resolve.

## 7 Limitations

Both datasets are single-time snapshots split randomly. A proper generalization test would use older data for training and newer phishing campaigns for testing; neither dataset is set up for that. Many high-importance features also require fetching and rendering the actual page, which introduces latency that would need to be managed in a real deployment.

The GRU used here is a basic adaptation of a sequential model to non-sequential tabular data. A multi-layer perceptron or attention-based model would be a more principled deep learning baseline. Finally, two datasets are better than one but not enough to make a strong generalization claim; data from multiple time periods and attack campaign types would give a more complete picture.

## 8 Conclusion

I compared Logistic Regression, Decision Tree, Random Forest, and a GRU on two phishing detection datasets. On PhiUSIIL all four hit near-perfect performance because the feature set makes classification nearly trivial. On UCI the real ranking appears: RF at  $F1 = 0.974$ , GRU at  $0.950$ , DT at  $0.944$ , and LR at  $0.917$ .

The word cloud of LR feature weights and the error analysis of UCI misclassifications both tell the same story: `URLSimilarityIndex` is the dominant signal on PhiUSIIL, and on UCI the hardest cases are pages that look structurally legitimate because attackers have learned to mimic the cues these models trust. A benchmark giving perfect scores to a linear model is too easy to reveal anything interesting about model differences. The harder dataset, and the specific cases that every model gets wrong, are where the useful findings actually are.



## References

- [1] Prasad, A., & Chandra, R. (2024). PhiUSIIL: Phishing URL/Website Identification using Intrinsic and Implicit Latent features. *Computers & Security*.
- [2] UCI ML Repository. PhiUSIIL Phishing URL Dataset (2024). <https://archive.ics.uci.edu/dataset/967/phiusiil+phishing+url+dataset>
- [3] UCI ML Repository. Phishing Websites Dataset (2015). <https://archive.ics.uci.edu/dataset/327/phishing+websites>
- [4] Tang, L., & Mahmoud, Q. H. (2021). A survey of machine learning-based solutions for phishing website detection. *Machine Learning and Knowledge Extraction*, 3(3), 672–694.
- [5] Zieni, R., Massari, L., & Calzarossa, M. C. (2023). Phishing or not phishing? A survey on the detection of phishing websites. *IEEE Access*, 11.
- [6] Whittaker, C., Ryner, B., & Nazif, M. (2010). Large-scale automatic classification of phishing pages. In *NDSS*.
- [7] Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. In *EMNLP*.
- [8] Anti-Phishing Working Group (2023). Phishing Activity Trends Report, Q4 2023. <https://apwg.org/trendsreports/>
- [9] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR*, 12, 2825–2830.
- [10] Paszke, A., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems* 32.
- [11] PyTorch Contributors. (2024). `torch.nn.GRU`. <https://pytorch.org/docs/stable/generated/torch.nn.GRU.html>
- [12] Mueller, A. C. (2012–2023). wordcloud: A little word cloud generator in Python. [https://amueller.github.io/word\\_cloud/](https://amueller.github.io/word_cloud/)